

# Music Playback I

Week 07.



COMT

SOFTWARE CLUSTER

// COMT • **COMPUTATIONAL THINKING** (CIT1C18)



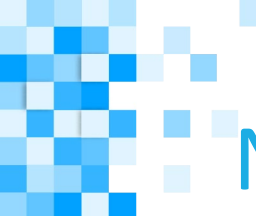
# Overview

- In this workshop, you will learn how to create the player screen and display the information of selected song on the screen.



# Objectives

- Demonstrate how to create Android UI;
- Demonstrate how to create Array;
- Demonstrate how to write methods and parameters.
- Demonstrate how to use Android Intent to receive data from another activity.



## Note

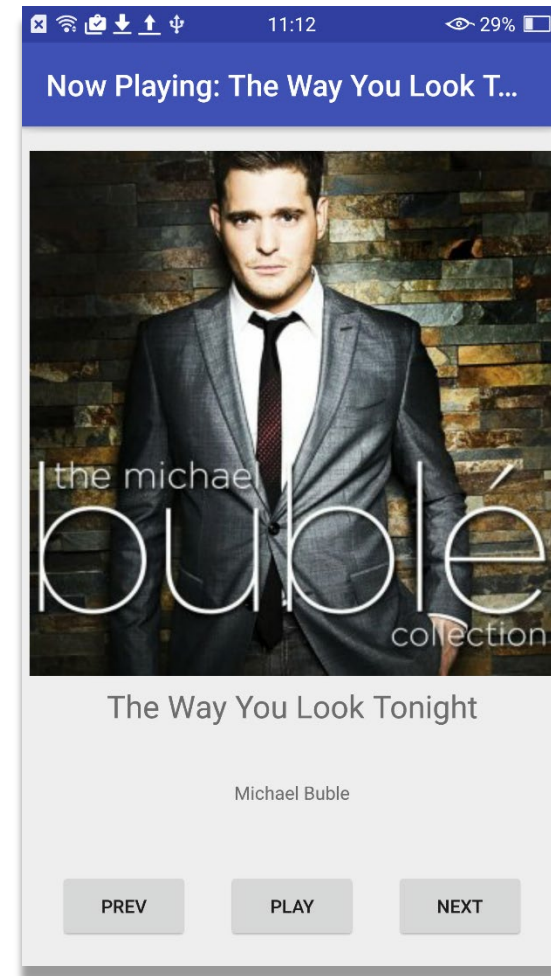
- Throughout this document, the acronym “**app**” will be used to refer to **mobile application**.



# CREATING THE PLAYBACK SCREEN

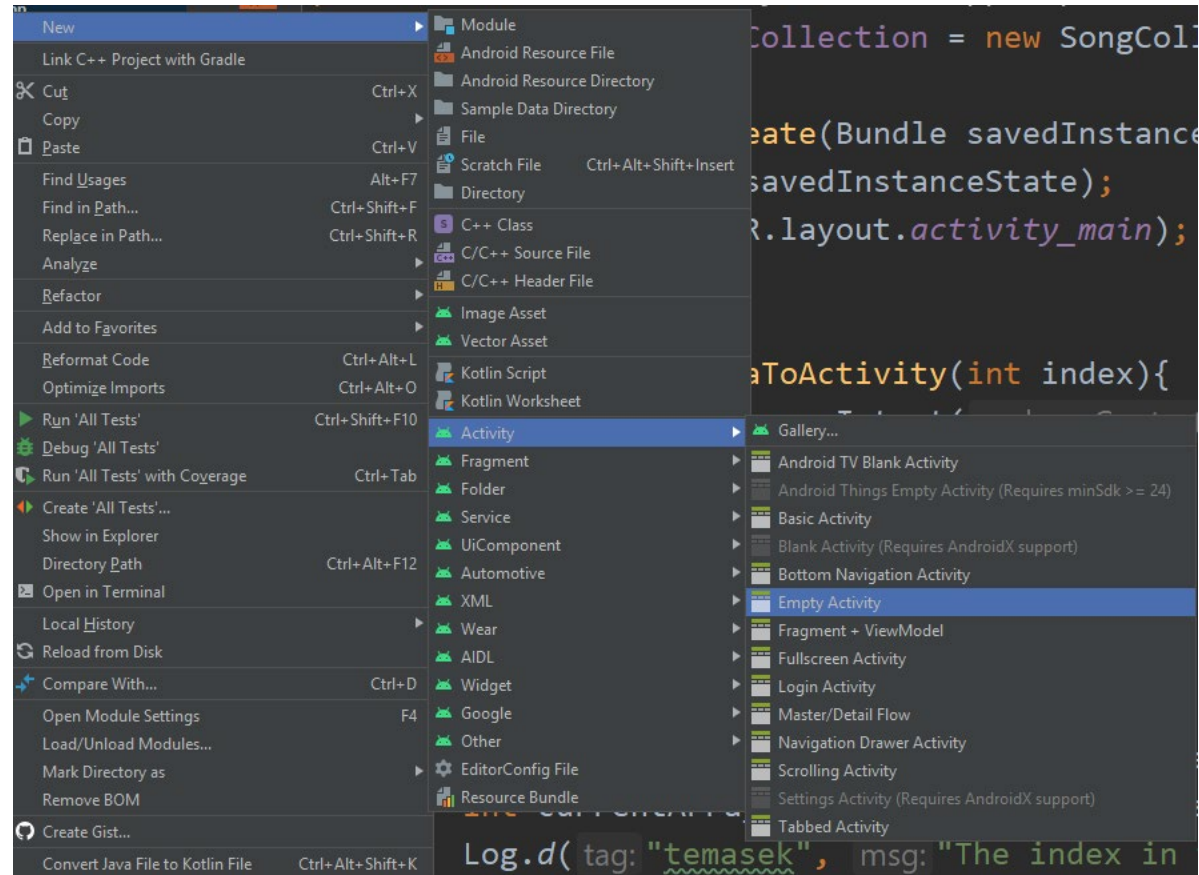
# Creating The Playback Screen

- In this lesson, you will be creating the playback screen as shown on the right.
- You will also learn how to receive the song information that was sent from the song list screen.



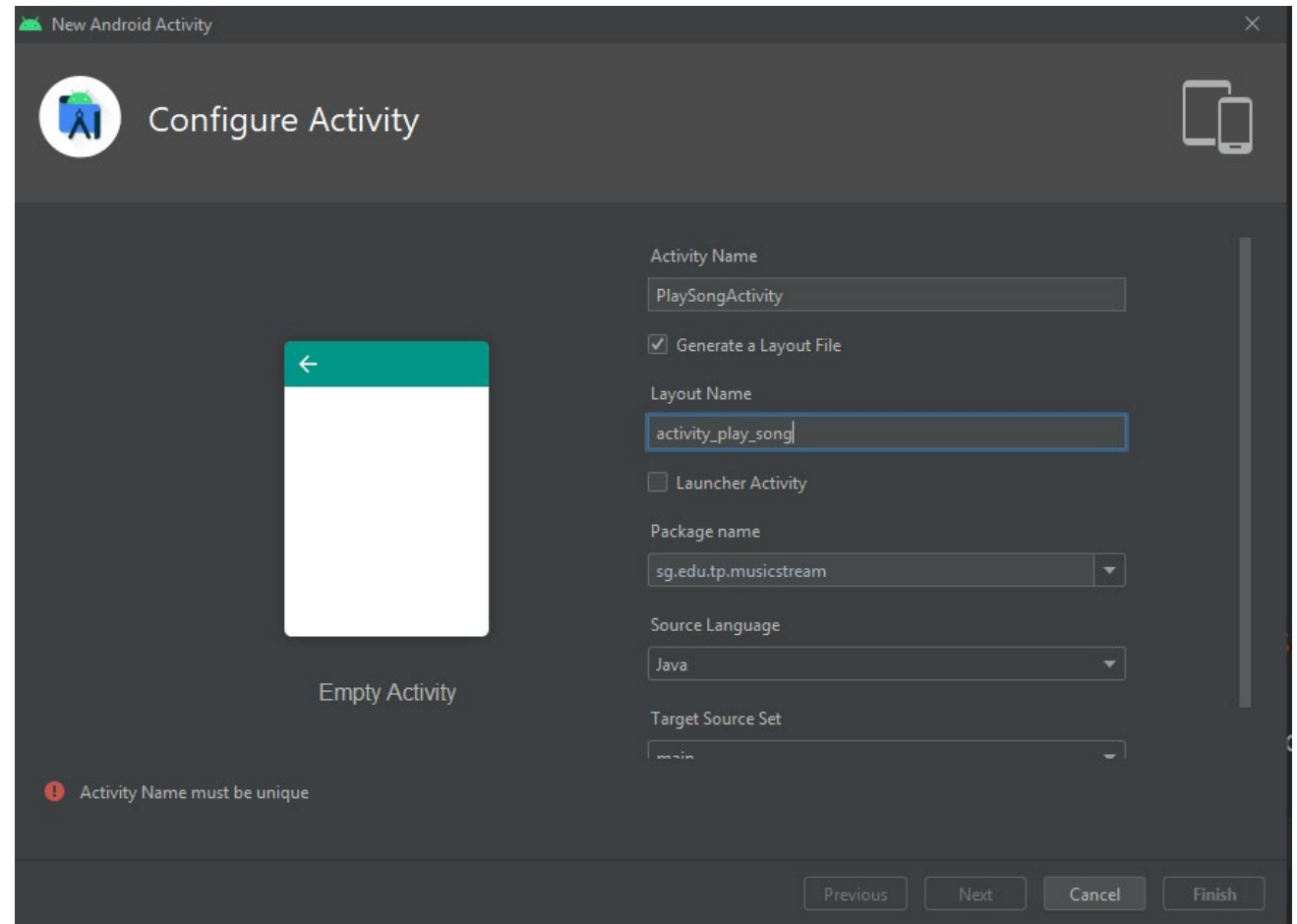
# Creating Playback Screen

- Open the **MusicStream** project in Android Studio.
- Right-click on the **app** folder and choose New -> Activity -> Empty Activity as shown in the diagram.



# Creating Playback Screen

- In the popup window, enter the **Activity Name** as `PlaySongActivity`.
- Ensure that **Generate Layout File** and **Backwards Compatibility** are both checked.
- Click on **Finish**.



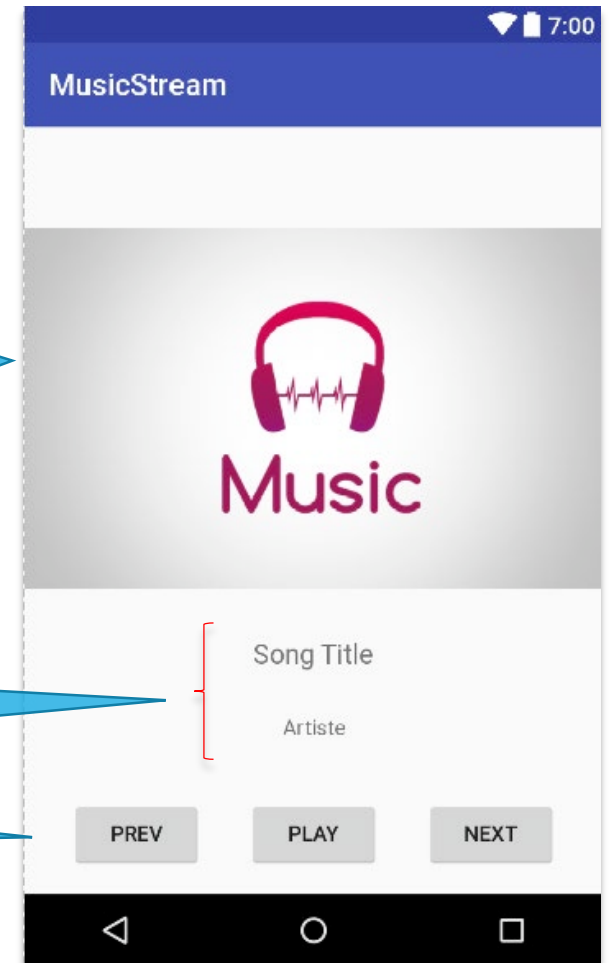
# Creating Playback Screen

- Once the new activity is created, two files are automatically created:
  - PlaySongActivity.java** (under **java->sg.edu.tp.musicstream** – your package name might differ and it is ok)
  - activity\_play\_song.xml** (under **res->layout**)
- Open the file **activity\_play\_song.xml** in design mode.
- Add the ui (user interface) according to this image.
- Refer to the next two slides for the properties that you need to set for each widget.

Create this cover art using an **ImageView**. Use **home.png** as the image.

Create song title and artiste name using **TextView**.

These three are **Button**.





# Creating Playback Screen

- Use the following information to help you add the widgets.

## Cover Art

| No. | Properties           | Value          |
|-----|----------------------|----------------|
| 1.  | <b>ID</b>            | imgCoverArt    |
| 2.  | <b>layout_width</b>  | 350dp          |
| 3.  | <b>layout_height</b> | 350dp          |
| 4.  | <b>srcCompat</b>     | @drawable/home |
| 5.  | <b>scaleType</b>     | fitCenter      |

## Song Title

| No. | Properties           | Value                                                                                 |
|-----|----------------------|---------------------------------------------------------------------------------------|
| 1.  | <b>ID</b>            | txtSongTitle                                                                          |
| 2.  | <b>layout_width</b>  | wrap_content                                                                          |
| 3.  | <b>layout_height</b> | wrap_content                                                                          |
| 4.  | <b>text</b>          | Song Title                                                                            |
| 5.  | <b>textSize</b>      | 24sp                                                                                  |
| 6.  | <b>textAlignment</b> |  |

## Artist Name

| No. | Properties           | Value                                                                                 |
|-----|----------------------|---------------------------------------------------------------------------------------|
| 1.  | <b>ID</b>            | txtArtist                                                                             |
| 2.  | <b>layout_width</b>  | wrap_content                                                                          |
| 3.  | <b>layout_height</b> | wrap_content                                                                          |
| 4.  | <b>text</b>          | Artist                                                                                |
| 5.  | <b>textSize</b>      | 14sp                                                                                  |
| 6.  | <b>textAlignment</b> |  |

# Creating Playback Screen

- Use the following information to help you add the widgets.

## Prev

| No. | Properties           | Value        |
|-----|----------------------|--------------|
| 1.  | <b>ID</b>            | btnPrevious  |
| 2.  | <b>layout_width</b>  | wrap_content |
| 3.  | <b>layout_height</b> | wrap_content |
| 4.  | <b>text</b>          | PREV         |
| 5.  | <b>onClick</b>       | playPrevious |

## Play

| No. | Properties           | Value            |
|-----|----------------------|------------------|
| 1.  | <b>ID</b>            | btnPlayPause     |
| 2.  | <b>layout_width</b>  | wrap_content     |
| 3.  | <b>layout_height</b> | wrap_content     |
| 4.  | <b>text</b>          | PLAY             |
| 5.  | <b>onClick</b>       | playOrPauseMusic |

## Next

| No. | Properties           | Value        |
|-----|----------------------|--------------|
| 1.  | <b>ID</b>            | btnNext      |
| 2.  | <b>layout_width</b>  | wrap_content |
| 3.  | <b>layout_height</b> | wrap_content |
| 4.  | <b>text</b>          | NEXT         |
| 5.  | <b>onClick</b>       | playNext     |



# RETRIEVING SONG DATA

# Retrieving the Array position from MainActivity

- After designing the playback screen, it is time to write the codes.
- Open the **PlaySongActivity.java** file.
- We will need to create some class variables so that we can use them later in the codes.
- We will also set some initial values to the variables.
- Add this set of codes as shown in the diagram.

```
1  package sg.edu.tp.musicstream;
2
3
4  import ...
5
16 public class PlaySongActivity extends AppCompatActivity {
17     private String title = "";
18     private String artiste = "";
19     private String fileLink = "";
20     private int drawable;
21     private int currentIndex = -1;
22
23     private MediaPlayer player = new MediaPlayer();
24     private Button btnPlayPause = null;
25     private SongCollection songCollection = new SongCollection();
26
27     @Override
28     protected void onCreate(Bundle savedInstanceState) {
29         super.onCreate(savedInstanceState);
```

# Retrieving index position from MainActivity

- Now, you will write the codes to retrieve the selected song information that was sent from the previous screen.
- Let's write the codes in the onCreate method as it will always be executed whenever the activity is being used

Ignore the codes with comments as they will only be used later

Do Log out in the console and try tapping different ImageButton to check the robustness of your codes

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_play_song);
    //btnPlayPause = findViewById(R.id.btnPlayPause);
    Bundle songData = this getIntent().getExtras();
    currentIndex = songData.getInt( key: "index");
    Log.d( tag: "temasek", msg: "Retrieved position is: " + currentIndex );
    //displaySongBasedOnIndex(currentIndex);
    //playSong(fileLink);
}
```

Recall that the array position was stored in a **Bundle** object/data type by the **Intent** to be sent to this screen/activity. This code shows how we can retrieve the array position from the **Intent** using the built-in **getInt** method.

Notice that we are using the same key "index" (specified in the sending codes previously written) to retrieve the individual values. The key must match in order to retrieve back the information

# Displaying Selected Song Data

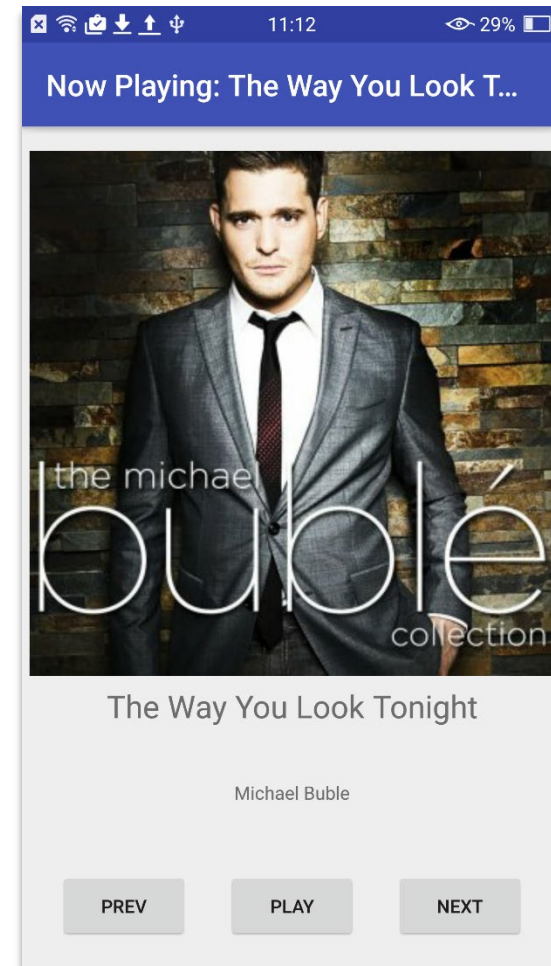
Now that we are able to get hold of the array position of the song. We need to use this information to find out the song title, artist etc. So we are going to create another method in the SongCollection class that will take in an argument (which is an integer), look up the position in the songs array and return to the caller that particular song instance.

For eg, if it is 0 being passed to this method, the method will return the first song instance which is the Michael bubble song.

```
public SongCollection(){  
    Song theWayYouLookTonight = new Song( id: "S1001",  
        title: "1. The Way You Look Tonight",  
        artiste: "Michael Buble",  
        fileLink: "https://p.scdn.co/mp3-preview/a5b8972e764025020625bbf9c1c2bbb06e394a60?cid=2afe87a64b0042dabf51f37318616965",  
        songLength: 4.66,  
        R.drawable.michael_buble_collection);  
  
    Song billieJean = new Song( id: "S1002",  
        title: "2. Billie Jean",  
        artiste: "Michael Jackson",  
        fileLink: "https://p.scdn.co/mp3-preview/4eb779428d40d579f14d12a9daf98fc66c7d0be4?cid=2afe87a64b0042dabf51f37318616965",  
        songLength: 4.9,  
        R.drawable.billie_jean);  
  
    Song ed = new Song( id: "S1003",  
        title: "3. One",  
        artiste: "Ed Sheeran",  
        fileLink: "https://p.scdn.co/mp3-preview/daa8679253ba20620db6e1db9c88edfc1f4069f?cid=2afe87a64b0042dabf51f37318616965",  
        songLength: 4.9,  
        R.drawable.photograph);  
  
    songs[0] = theWayYouLookTonight;  
    songs[1] = billieJean;  
    songs[2] = ed;  
}  
  
public Song getCurrentSong(int currentSongId){  
    return songs[currentSongId];  
}  
  
public int searchSongById(String id){  
    for(int index=0; index < songs.length; index++){  
        Song tempSong = songs[index];  
        if(tempSong.getId().equals(id)){  
            return index;  
        }  
    }  
    return -1;  
}
```

# Displaying Selected Song Data

- After the ability to get the Song info, the next important step to do now is to display the selected song data on the screen such as the one shown here.
- We will create a new method in the PlaySongActivity called `displaySongBasedOnIndex`.
- The task for this method is to display the cover art, song title, and artist name correctly in the layout



# Displaying Selected Song Data

- The codes for `displaySongBasedOnIndex` method is shown below.
- In order to set each song data to the corresponding widget in the playback screen, we need to retrieve the widget object using its ID. This is done by using a built-in method called `findViewById()`.

```
public void displaySongBasedOnIndex(int selectedIndex) {  
    Song song = songCollection.getCurrentSong(currentIndex);  
    title = song.getTitle();  
    artiste = song.getArtiste();  
    fileLink = song.getFileLink();  
    drawable = song.getDrawable();  
    TextView txtTitle = findViewById(R.id.txtSongTitle);  
    txtTitle.setText(title);  
    TextView txtArtiste = findViewById(R.id.txtArtiste);  
    txtArtiste.setText(artiste);  
    ImageView iCoverArt = findViewById(R.id.imgCoverArt);  
    iCoverArt.setImageResource(drawable);  
}
```

These are class variables that we had created previously. These variables can be referenced anywhere in the `PlaySongActivity` class

Recall the earlier method `getCurrentSong` that we had created 2 slides before? Here we are calling it so that we can get the Song information by passing in the position in the array

Setting `txtArtiste` and using `findViewById` to point to the actual ui which is the textview for the artiste name

Changing the text content of the `txtArtist` textview

`setImageResource` will change the image in the `Imageview` ui

# Retrieving index position from MainActivity

- We can now uncomment the codes in the onCreate method as we had now created the displaySongBasedOnIndex method in PlaySongActivity

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_play_song);
    btnPlayPause = findViewById(R.id.btnPlayPause);
    Bundle songData = this.getIntent().getExtras();
    currentIndex = songData.getInt( key: "index");
    Log.d( tag: "temasek", msg: "Retrieved position is: " + currentIndex );
    displaySongBasedOnIndex(currentIndex);
    //playSong(fileLink);
}
```

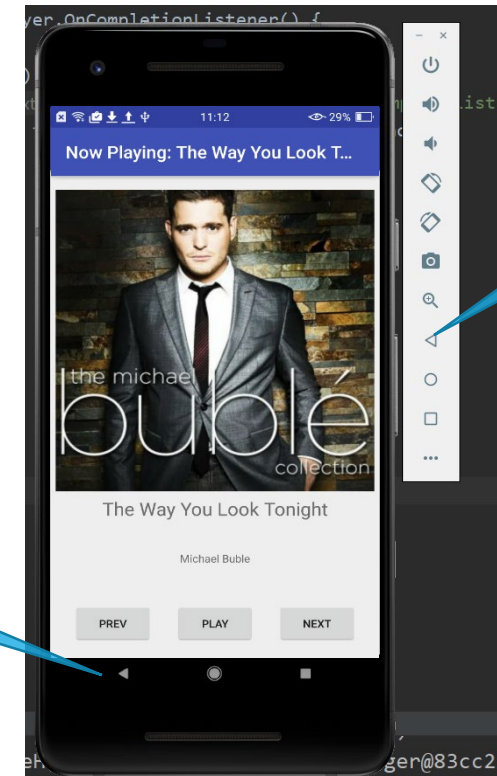
Ignore this code with comments as we do not have the playSong method yet

Calling the displaySongBasedOnIndex method so that the various ui will be updated with the correct information

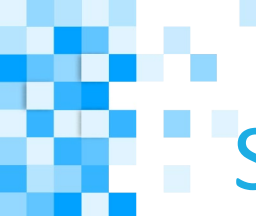
# Summary

- You can now try to run your app in the emulator and see that the song information is updated in the PlaySongActivity
- You can press the back button of the emulator and tapped on another ImageButton
- In the next lesson, you will learn how to play the selected song.

Back Button



Back Button



# Supplementary Materials

| Topic                   | Source                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java<br>Method/Function | <a href="https://www.learnjavaonline.org/en/Functions">https://www.learnjavaonline.org/en/Functions</a><br><a href="https://mathbits.com/MathBits/Java/Methods/Lesson1.htm">https://mathbits.com/MathBits/Java/Methods/Lesson1.htm</a><br><a href="http://www.horsesnw.com/articles/JavaMethod.htm">http://www.horsesnw.com/articles/JavaMethod.htm</a>                      |
| Android Intent          | <a href="https://developer.android.com/training/basics/firstapp/starting-activity.html#BuildIntent">https://developer.android.com/training/basics/firstapp/starting-activity.html#BuildIntent</a><br><a href="http://www.vogella.com/tutorials/AndroidIntent/article.html#what-are-intents">http://www.vogella.com/tutorials/AndroidIntent/article.html#what-are-intents</a> |

# THE END



# COMT